- **TOPIC'S:**

  1) ELEMENT'S OF CACHE DESIGN.

  2) SINGLE CYCLE PROCESSOR.

  3) MULTI CYCLE PROCESSOR.

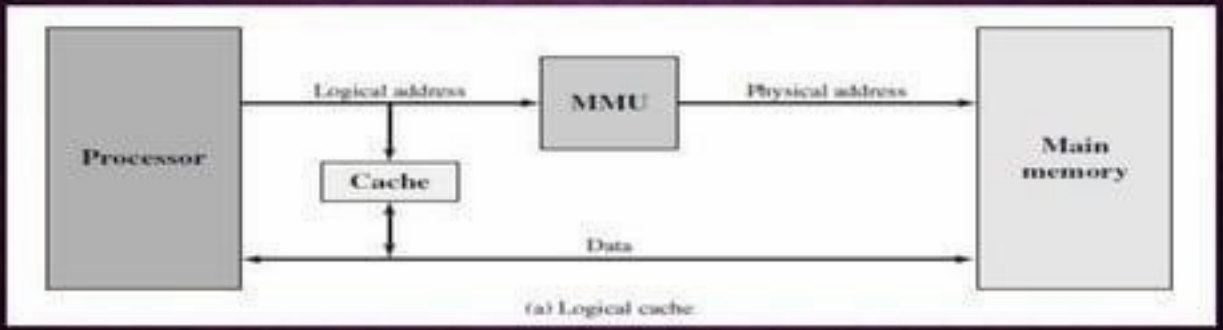
ELEMENT'S OF CACHE DESIGN

- Cache Addresses
- Cache Size
- Mapping Function
- Replacement Algorithm
- Write Policy
- Line Size
- Number Of Caches

CACHE ADDRESSES

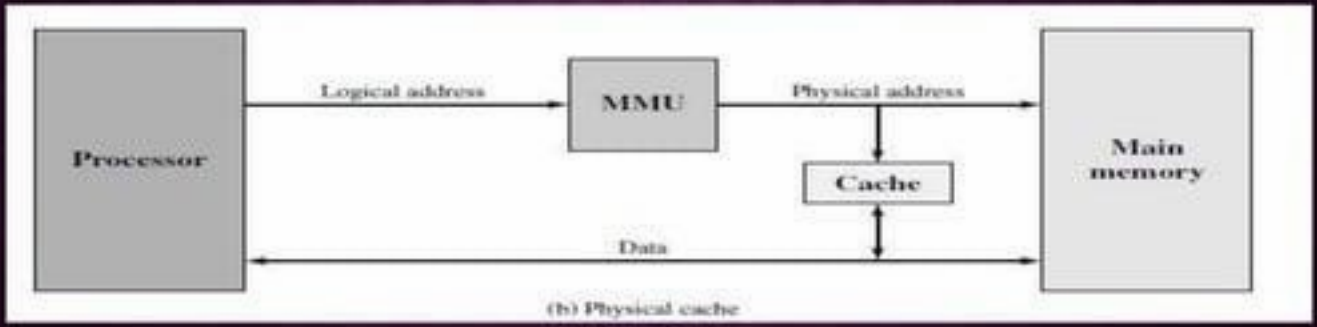
- Logical Address:

  When virtual addresses are used, the cache can be placed between the processor and the MMU or between the MMU and main memory. A logical cache, also known as a virtual cache, stores data using virtual addresses. The processor accesses the cache directly, without going through the MMU.



- Physical Address:

  A physical cache stores data using main memory physical addresses. One advantage of the logical cache is that cache access speed is faster than for a physical cache, because the cache can respond before the MMU performs an address translation.



CACHE SIZE

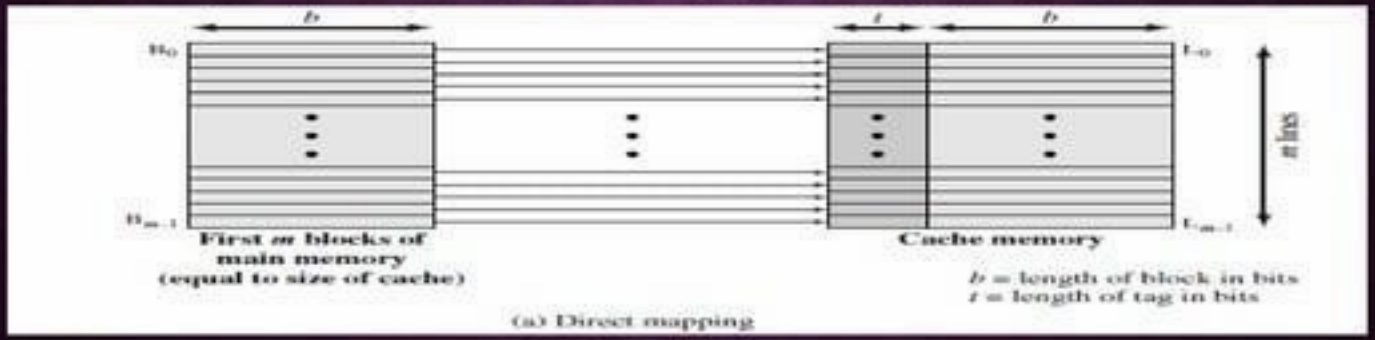
The size of the cache should be small enough so that the overall average cost per bit is close to that of main memory alone and large enough so that the overall average access time is close to that of the cache alone.

MAPPING FUNCTION

As there are fewer cache lines than main memory blocks, an algorithm is needed for mapping main memory blocks into cache lines. Further, a means is needed for determining which main memory block currently occupies a cache line. The choice of the mapping function dictates how the cache is organized. Three techniques can be used: direct, associative, and set associative.

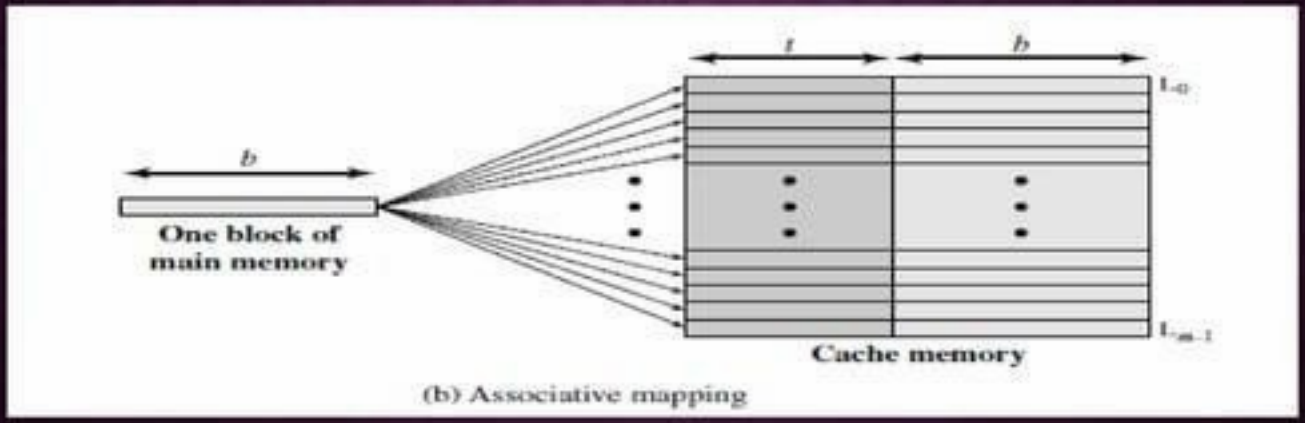
- **DIRECT MAPPING:**

  The simplest technique, known as direct mapping, maps each block of main memory into only one possible cache line.



- **ASSOCIATIVE MAPPING:**

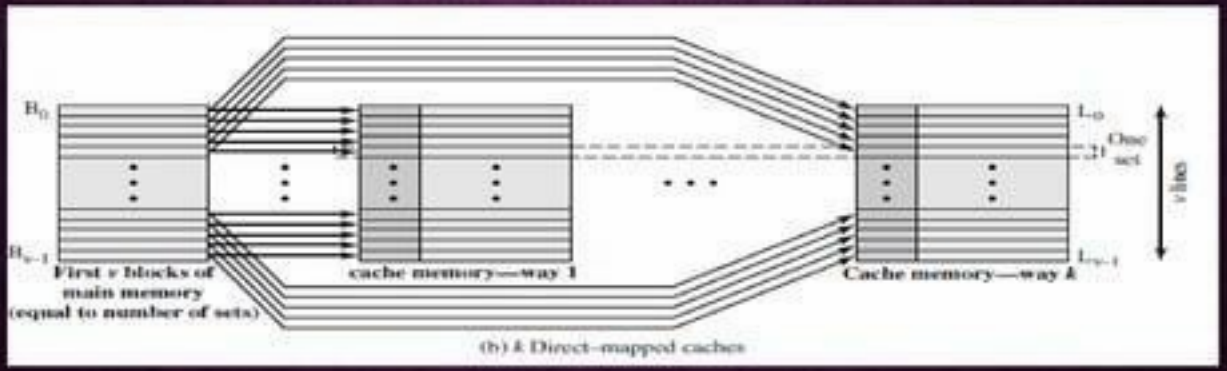
Associative mapping overcomes the disadvantage of direct mapping by permitting each main memory block to be loaded into any line of the cache.



(b) Associative mapping

- **SET-ASSOCIATIVE MAPPING:**

Set-associative mapping is a compromise that exhibits the strengths of both the direct and associative approaches. With set-associative mapping, block can be mapped into any of the lines of set j.



(b) k Direct-mapped caches

REPLACEMENT ALGORITHM'S

Once the cache has been filled, when a new block is brought into the cache, one of the existing blocks must be replaced. For direct mapping, there is only one possible line for any particular block, and no choice is possible. For the associative and set associative techniques, a replacement algorithm is needed. To achieve high speed, such an algorithm must be implemented in hardware.

- Least Recently Used (LRU):

  Implement in L1 cache and use for replacing recently activities.

- Least Frequently Used(LFU):

  Implement in L2 cache and use for replacing frequently activities.

- First In First Out (FIFO):

  Implement in L3 cache and use for replacing long term activities.

WRITE POLICY

- Write Through:

  The information is written to both block in cache and the block in lower-level memory.

- Write Back:

  The information is written only to block in cache. The modified cache block is written to main memory only when it is replaced.

- Dirty Bit:

  It is indicate whether the block is dirty (modified while in cache) .

- Cache Hit:

  A memory reference where the required data is found in cache.

- Cache Miss:

  A memory reference where the required data is not found in cache.

LINE SIZE

- No of Lines increase no of reference increases. When a block of data is retrieved and placed in the cache, not only the desired word but also some number of adjacent words are retrieved. As the block size increases from very small to larger sizes, the hit ratio will at first increase because of the principle of locality, which states that data in the vicinity of a referenced word are likely to be referenced in the near future.

NUMBER OF CACHES

- MULTILEVEL CACHES:

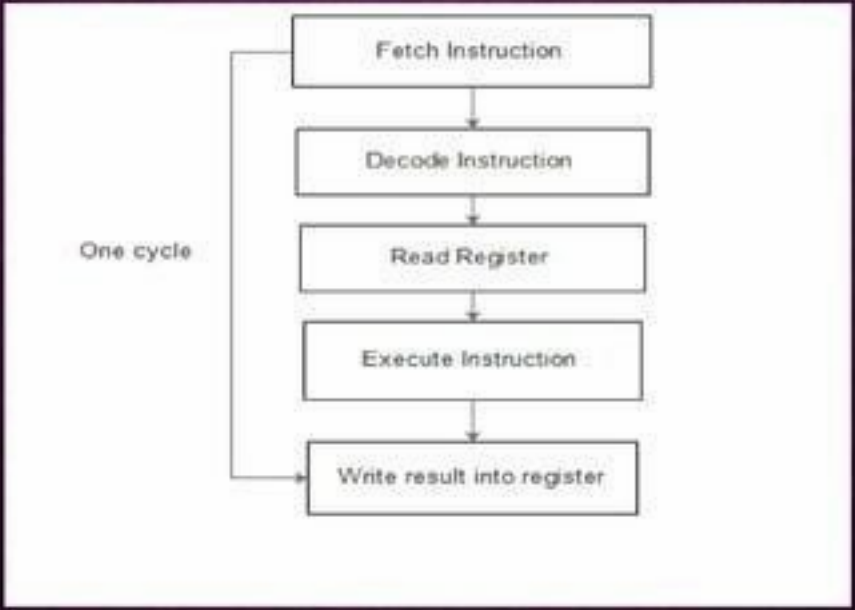
Most contemporary designs include both on-chip and external caches. The simplest such organization is known as a two-level cache, with the internal cache designated as level 1 (L1) and the external cache designated as level 2 (L2). There can also be 3 or more levels of cache. This helps in reducing main memory accesses.

- UNIFIED VERSUS SPLIT CACHES:

Earlier on-chip cache designs consisted of a single cache used to store references to both data and instructions. This is the unified approach. More recently, it has become common to split the cache into two: one dedicated to instructions and one dedicated to data. These two caches both exist at the same level. This is the split cache. Using a unified cache or a split cache is another design issue.

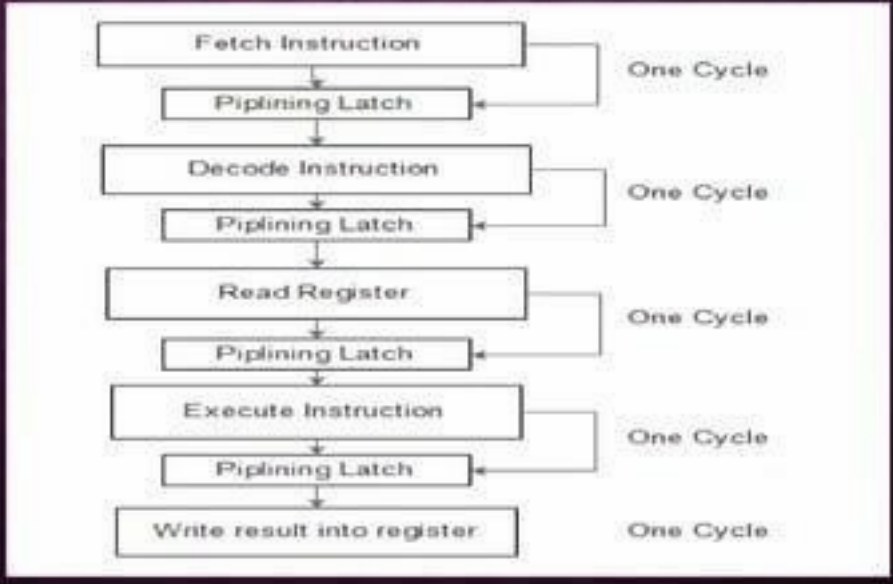
SINGLE CYCLE PROCESSOR

A processor that carries out one instruction in a single clock cycle.



MULTI CYCLE PROCESSOR

Multi-cycle processors break up the instruction into its fundamental parts, and executes each part of the instruction in a different clock cycle. Since signals have less distance to travel in a single cycle, the cycle times can be sped up considerably.



Two-Level Cache Performance Characteristics

Modern CPU memory hierarchy often uses **L1** and **L2** caches to reduce average memory access time (AMAT). Performance depends heavily on **locality of reference** and **efficient cache operations**.

1. Locality of Reference

Locality describes how programs access memory. Two-level caches exploit both **temporal** and **spatial** locality.

a. Temporal Locality

- Programs tend to reuse recently accessed data or instructions.
- L1 cache is designed small and very fast to maximize hits from temporal locality.

- L2 cache (larger but slower than L1) holds additional recently used blocks to catch accesses that miss in L1 but may still be reused.

Impact on performance:

- High temporal locality → high L1 hit rate → minimal latency.
- Misses in L1 but hits in L2 still save time compared to main memory.

b. Spatial Locality

- Programs often access memory locations close to each other (e.g., iterating arrays).
- Caches use **cache lines/blocks** to bring in contiguous bytes.

Role in two-level caches:

- L1 has smaller block size for reduced miss penalty.
- L2 has slightly larger blocks to reduce compulsory misses and capture broader spatial locality.

Impact on performance:

- Improves prefetch effectiveness.
- Reduces compulsory and capacity misses.

c. Types of Locality Misses

- **Compulsory misses**: First-time accesses; often serviced by L2 faster than main memory.
- **Capacity misses**: Occur when working set exceeds cache size; L2 reduces them due to larger size.
- **Conflict misses**: Depend on associativity; higher associativity in L2 reduces these.

2. Cache Operations in Two-Level Caches

a. Cache Hit and Miss Handling

1. **L1 Hit**
   - Fastest access (1–4 cycles typically).
   - Data returned immediately.
2. **L1 Miss → L2 Access**
   - L2 handles request.
   - If **L2 Hit**: data returned to L1 (and L1 updated).
   - If **L2 Miss**: fetch from main memory → update L2 → update L1.

Average Memory Access Time (AMAT):

AMAT=tL1+(miss rateL1)(tL2+miss rateL2×tmemory)AMAT = t_{L1} + (miss\ rate_{L1})(t_{L2} + miss\ rate_{L2}\times t_{memory})AMAT=tL1+(miss rateL1)(tL2+miss rateL2×tmemory)

b. Write Operations

Two main policies:

1. Write-Through vs Write-Back

- **L1 often write-back**: reduces write latency, writes propagated to L2 on eviction.
- **L2 may be write-through to memory** for consistency.

2. Write-Allocate vs No-Write-Allocate

- **Write-Allocate** (common with write-back)
    - On a write miss, block is brought into cache, then written.
- **No-Write-Allocate** (common with write-through)
    - Missed writes go directly to lower levels without loading block into cache.

c. Inclusive vs Exclusive Cache Policies

- **Inclusive**: L2 contains everything in L1
    - Simplifies coherence.
    - Increases pressure on L2 capacity.
- **Exclusive**: L1 and L2 contain disjoint data

    - Maximizes total cache capacity.
    - Slightly more complex data movement.
- **Non-inclusive/Non-exclusive**: hybrid.

d. Replacement Policies

Used when a cache is full:

- **LRU (Least Recently Used)** — common in L1
- **Pseudo-LRU** — used in L2 for reduced complexity
- **Random** — simple, sometimes used in large L2 caches

These affect conflict miss rates and therefore performance.

e. Prefetching

- L2 often performs **aggressive prefetching**, exploiting spatial locality.

- Reduces miss latency but risks cache pollution.

3. Summary Table

Feature	L1 Cache	L2 Cache
Speed	Very fast	Slower
Size	Small	Larger
Exploits	Temporal locality	Both temporal + spatial locality
Miss penalty	Low	Higher but less than main memory
Replacement	Usually LRU	Pseudo-LRU / Random
Write policy	Often write-back	Write-back or write-through